OOP Lab Examination – 40 Marks

Question Code: BOB

Duration: 2 Hours

Problem Statement: Travel Package Management System

Develop a Java application for a **Travel Package Management System** for a travel company that provides different types of holiday packages.

The system should allow the company to store package details, calculate the total package cost, apply discounts, and display the final payable amount.

1. Create an Abstract Class TravelPackage – 8 Marks

Create an abstract class named `TravelPackage` with the following instance variables:

- packageId
- customerName
- destination
- numberOfDays
- numberOfPersons

Create a parameterized constructor to initialize all the above variables.

Create a concrete method:

```
displayPackageDetails()
```

to display the common package information.

Declare the following abstract method:

```
calculateCost()
```

The method should be implemented differently by the child classes.

Also declare a static variable:

```
totalPackages
```

This variable should keep track of the total number of package objects created.

2. Create DomesticPackage Class – 7 Marks

Create a class named `DomesticPackage` that extends the `TravelPackage` class.

The class should contain:

- hotelName
- transportMode
- mealPlan
- costPerPerson

Initialize all variables using a constructor.

Override the `calculateCost()` method.

The package cost should be calculated as:

```
cost = costPerPerson × numberOfPersons
```

Display all domestic-package-specific details along with the common package details.

3. Create InternationalPackage Class – 7 Marks

Create a class named `InternationalPackage` that extends the `TravelPackage` class.

The class should contain:

- hotelName
- flightClass
- visaRequired
- costPerPerson

Initialize all variables using a constructor.

Override the `calculateCost()` method.

The package cost should be calculated as:

```
cost = costPerPerson × numberOfPersons
```

Display all international-package-specific details along with the common package details.

4. Create an Interface DiscountApplicable – 6 Marks

Create an interface named `DiscountApplicable` containing:

```
calculateDiscount(double cost)
```

Both `DomesticPackage` and `InternationalPackage` classes should implement this interface.

Apply different discount rules.

Domestic Package:

- If number of persons is 4 or more, provide a 10% discount.
- Otherwise, no discount.

International Package:

- If number of persons is 3 or more, provide a 7% discount.
- Otherwise, no discount.

The method should return the discount amount.

5. Static Variable – 3 Marks

The static variable `totalPackages` in the `TravelPackage` class should be incremented whenever a new package object is created.

At the end of the program, display:

```
Total Packages Created: <value>
```

The value should represent the total number of `DomesticPackage` and `InternationalPackage` objects created.

6. Main Method and Object Creation – 9 Marks

Create a class `TravelPackageDemo` containing the `main()` method.

Create at least:

- 2 DomesticPackage objects
- 2 InternationalPackage objects

For every package, display:

1. Package ID
2. Customer name
3. Destination
4. Number of days
5. Number of persons
6. Package-specific details
7. Total cost
8. Discount
9. Final payable amount

The final payable amount should be calculated as:

```
Final Amount = Total Cost - Discount
```

Finally, display the value of:

```
totalPackages
```

Java Lab Examination – ArrayList – 40 Marks

Question Code: IOB

Duration: 2 Hours

Problem Statement: Vehicle Rental Management System

Develop a Java application for a **Vehicle Rental Management System** using `ArrayList`.

Create a class `VehicleRental` with the following instance variables:

- rentalId
- vehicleNumber
- customerName
- vehicleType
- rentalType
- rentalCost
- rentalStatus

Use an:

```
ArrayList<VehicleRental>
```

to store all rental records.

Possible values:

Vehicle Type:

- Car
- Bike
- SUV

Rental Type:

- Daily
- Weekly
- Monthly
- Long Term

Rental Status:

- Reserved
- Active
- Completed

- Cancelled

1. Add and Display Rental Records – 8 Marks

Implement:

```
addRental()
displayAllRentals()
```

`addRental()` should create a `VehicleRental` object and add it to the `ArrayList`.

The `rentalId` must be unique.

`displayAllRentals()` should display all rental records in a meaningful format.

2. Search Rental Record – 5 Marks

Implement:

```
searchByVehicleNumber()
```

The user should enter a vehicle number.

Display the complete rental details of the matching vehicle.

If no record is found, display an appropriate message.

The search must be performed using the `ArrayList`.

3. Update Rental Record – 5 Marks

Implement:

```
updateRental()
```

The user should enter a `rentalId`.

If the rental record exists, allow the user to update:

- rentalType
- rentalCost
- rentalStatus

If the rental ID does not exist, display an appropriate message.

4. Remove Rental Record – 4 Marks

Implement:

```
removeRental()
```

The user should enter a `rentalId`.

If the record exists, remove it from the `ArrayList`.

Otherwise, display:

```
Rental record not found.
```

5. Sort Rental Records – 5 Marks

Implement:

```
sortByRentalCost()
```

Sort the rental records based on `rentalCost` in **descending order** using `Comparator`.

After sorting, display the records.

The highest rental cost should appear first.

6. Business Logic – 8 Marks

Implement:

```
generateRentalReport()
```

The method should analyze all rental records and display:

1. Total revenue generated from completed rentals.
2. Vehicle number of the rental having the highest rental cost.

Display the report in a meaningful format.

Java Lab Examination – File Handling – 40 Marks

Question Code: PNB

Duration: 2 Hours

Problem Statement: Movie Ticket Booking Management System

Develop a Java application for a **Movie Ticket Booking Management System** using **object-based file handling**.

The application should maintain movie ticket booking details using serialized Java objects. The system should allow the user to add, display, search, update, delete, and generate a booking report.

Create a class named `MovieTicket` that implements `Serializable`.

The class should contain the following instance variables:

- `bookingId`
- `customerName`
- `movieName`
- `theatreName`
- `showTime`
- `numberOfTickets`
- `ticketPrice`
- `bookingStatus`

Possible values for `bookingStatus`:

- Booked
- Confirmed
- Cancelled
- Completed

---

1. Create MovieTicket Class – 5 Marks

Create a class named:

`MovieTicket`

The class must:

- Implement `Serializable`
- Contain all the required instance variables
- Have a parameterized constructor
- Provide a method to display booking details

Each booking must have a unique `bookingId`.

---

2. Add and Write Object to File – 7 Marks

Create a method:

`addBooking()`

The method should:

1. Accept booking details from the user.
2. Create a `MovieTicket` object.
3. Write the object into the file.

Use:

`ObjectInputStream`

and

`ObjectOutputStream`

The booking data must remain available even after the application is closed.

Before adding a new booking, ensure that the `bookingId` is unique.

---

3. Read and Display Objects – 5 Marks

Create a method:

`displayAllBookings()`

Read all `MovieTicket` objects from the file and display their complete details.

If there are no booking records, display an appropriate message.

---

4. Search Booking – 5 Marks

Create a method:

`searchBooking()`

The user should enter a `bookingId`.

Read the objects from the file and search for the matching booking.

If the booking is found, display the complete booking details.

If the booking is not found, display:

`Booking not found.`

---

5. Update Booking – 5 Marks

Create a method:

`updateBooking()`

The user should enter a `bookingId`.

If the booking exists, allow the user to update:

- `showTime`
- `numberOfTickets`
- `ticketPrice`
- `bookingStatus`

After updating, rewrite the complete collection of booking objects to the file.

If the booking ID does not exist, display an appropriate message.

---

6. Delete Booking – 4 Marks

Create a method:

`deleteBooking()`

The user should enter a `bookingId`.

If the booking exists:

- Remove the corresponding object.
- Rewrite the remaining booking objects to the file.

If the booking does not exist, display:

```
Booking not found.
```

---

7. Business Logic – 5 Marks

Create a method:

```
generateBookingReport()
```

Read all `MovieTicket` objects from the file and display:

1. Total revenue generated from **Confirmed** bookings.
2. The booking having the **highest number of tickets**.
3. The movie name for which the highest number of tickets were booked.

The total booking amount should be calculated as:

```
Booking Amount = numberOfTickets × ticketPrice
```

Display the report in a meaningful format.

Example:

```
===== MOVIE BOOKING REPORT =====

Confirmed Bookings       : 8
Total Booking Revenue    : ₹18,500
Highest Ticket Booking   : B105
Movie Name               : Action Warriors
Number of Tickets        : 6
```

The calculations must be performed using the objects read from the file.

---

8. Main Method and Menu – 4 Marks

Create a class:

```
MovieTicketManagement
```

with the `main()` method.

Display the following menu:

```
===== MOVIE TICKET BOOKING MANAGEMENT =====

1. Add Booking
2. Display All Bookings
3. Search Booking
4. Update Booking
5. Delete Booking
```

```
6. Generate Booking Report
7. Exit
```

The menu should continue until the user selects **Exit**.

Sample Data

Students may use the following data for testing:

Booking ID	Customer Name	Movie Name	Theatre Name	Show Time	Tickets	Ticket Price	Status
B101	Arun	Action Warriors	PVR Orion	10:30 AM	2	250	Confirmed
B102	Meena	Hidden Truth	INOX Garuda	2:00 PM	4	300	Booked
B103	Ravi	Future World	PVR Forum	6:30 PM	3	350	Confirmed
B104	Priya	Love Beyond Time	INOX Mantri	9:00 PM	5	220	Completed
B105	Karthik	Action Warriors	PVR Orion	6:30 PM	6	250	Confirmed

---

JDBC Lab Examination – 40 Marks

Question Code: CUB

Duration: 2 Hours

Problem Statement: Hotel Reservation Management System

Develop a **Java JDBC application** for a **Hotel Reservation Management System**.

A hotel maintains details of rooms reserved by customers. The application should allow the user to add, modify, delete, search, and display reservation records using **JDBC and MySQL**.

Create a database named:

`hotel_reservation`

Create a table named:

`reservation`

with the following columns:

Column	Data Type	Description
reservation_id	INT	Primary Key
guest_name	VARCHAR(50)	Guest name
room_number	VARCHAR(10)	Unique room number
room_type	VARCHAR(30)	Standard / Deluxe / Suite
stay_days	INT	Number of days
room_rate	DOUBLE	Rate per day
status	VARCHAR(20)	Reserved / Checked-In / Checked-Out

1. Database and Table Creation – 4 Marks

Create the `hotel_reservation` database and `reservation` table.

Set `reservation_id` as the **Primary Key**.

The `room_number` should be unique for an active reservation.

2. Insert Reservation – 6 Marks

Implement:

`addReservation()`

Accept reservation details from the user and insert a new record into the `reservation` table.

The application should use `PreparedStatement` for the INSERT operation.

Before insertion, ensure that the room number is not already registered for an active reservation.

Display an appropriate success or error message.

3. Update Reservation – 5 Marks

Implement:

`updateReservation()`

Ask the user to enter `reservation_id`.

If the reservation exists, allow the user to update:

- room_type
- stay_days
- room_rate
- status

Use a parameterized `UPDATE` query.

Display an appropriate message if the specified reservation ID does not exist.

4. Delete Reservation – 5 Marks

Implement:

`deleteReservation()`

Ask the user to enter `reservation_id`.

If the reservation exists, delete the corresponding record from the database.

If the ID does not exist, display:

`Reservation not found.`

Use a parameterized `DELETE` query.

5. Select Reservation by ID – 5 Marks

Implement:

`findReservationById()`

Ask the user to enter `reservation_id`.

Retrieve and display the complete details of the reservation.

If no record exists for the given ID, display an appropriate message.

Use:

`PreparedStatement`

and

`ResultSet`

6. Select All Reservations – 5 Marks

Implement:

`displayAllReservations()`

Retrieve all reservation records from the database and display them in a meaningful tabular format.

7. Business Logic – 6 Marks

Implement:

`calculateBill()`

Ask the user to enter a `reservation_id`.

Retrieve the reservation details and calculate:

`Total Bill = stay_days × room_rate`

Display the guest name, room number, number of days, room rate, and total bill.

If the reservation does not exist, display an appropriate message.

8. Main Method and Menu – 4 Marks

Create a class:

`HotelReservationManagement`

containing the `main()` method.

Display:

```
===== HOTEL RESERVATION MANAGEMENT =====

1. Add Reservation
2. Update Reservation
3. Delete Reservation
4. Find Reservation by ID
5. Display All Reservations
6. Calculate Bill
7. Exit
```

The program should continue displaying the menu until the user selects **Exit**.

Sample Data

Students may insert the following records for testing:

ID	Guest Name	Room Number	Room Type	Days	Rate	Status
201	Anil	R101	Standard	3	1800	Checked-In
202	Priya	R205	Deluxe	5	2800	Reserved
203	Karthik	R310	Suite	2	4500	Checked-Out
204	Meena	R108	Standard	4	1800	Reserved
205	Ravi	R402	Suite	6	5000	Checked-In

.